

Python: Files

Cheng-Te Li (李政德)

chengte@mail.ncku.edu.tw

Dept. of Statistics, NCKU



Why Coding?

- 服務生學寫程式 年薪62萬暴增至310萬
<http://bit.ly/2fL98Ke>
- 企管女碩士轉職到寫程式的生涯
<http://bit.ly/2fgUhCW>
- 籃球員也會 Coding !
NBA 巨星 Chris Bosh 告訴你，Coding 如何影響他的一生
<https://buzzorange.com/techorange/2013/11/08/nba-coding-chris-bosh/>
- 白天走台步，晚上寫程式：世界頂級時尚名模的雙面生活
<http://www.inside.com.tw/2014/01/06/from-coding-to-the-catwalk-this-high-fashion-model-has-a-secret-double-life>
- 記者為什麼要學寫程式？
<https://www.twreporter.org/a/opinion-data-journalist>
- 醫師變工程師，下海寫程式幫助推廣家醫制度
<http://www.ithome.com.tw/news/89622>

File Manipulation

How to work with files in Python?

Electronic files



Files store useful data
in an meaningful easy
way to find them (organized).

File Manipulation

- Files: store large amount of data in an organized way for efficient retrieval. Make data persistent in the disk!
- Types of files
 - General Texts (ex: .txt, .csv, .dat)
 - Structured Texts (ex: .html, .json, .xml)
 - Multimedia (ex: .jpg, .png, .mp3, .avi, .mp4, .wmv)
 - Database (ex: .sql, .db, .zip)
- Why text files?
 - Text files are cross-platform (same on Windows, Linux, Mac)
 - Text files are easy to use
 - Very common for data analysis

I/O Text Files

- As far as Python is concerned, **a file is just a string** (often very large!) stored on your file system, that you can read or write, gradually or all together
- When you usually do with files
 - **Open** the file (by double clicking the icon)
 - Let your PC to **Read** some files
 - Do some editing and **Write** the new data into files
 - **Close** the file (by clicking the X on the top-right)
- **Sequential file processing**
 - Contain a linear sequence of characters
 - Or separated into lines
 - The **Open-Read-Close** idiom

Python's Text-File Model (Read File)

- Typical code skeleton:

```
file_name = "whatever.txt"
tf = open(file_name, "r") # (1) Must "open" before use
# (2) Read through the file from beginning to end
tf.close()                # (3) "Close" when done
```

- Variable `file_name`: the string of filename or file path
- Specify file mode when opening: `open(filename, mode)`
 - `"r"` is for **reading**, `"w"` is for **writing**, `"a"` is for **appending**
- Variable `tf` points to file object
- **Need to close a file after we are done:** `f.close()`
- When you open a file, **the reading cursor is positioned on the first character of the file.**

So, you can read data from the beginning of the file

Python's File-Reading Operations

- Functions available for reading the files:
(1) `read()`, (2) `readline()` and (3) `readlines()`
- `read()` – this function reads all characters as a big string, unless you specify otherwise by giving the number of characters

```
file_name = "test.txt"
myfile = open(file_name, "r")
text1 = myfile.read()      # read the entire file as a string
text2 = myfile.read(10)    # read the first 10 characters
myfile.close()
```

Python's File-Reading Operations

- `readline()` – this function reads a single line from file
- `readlines()` – this function returns a list containing all the lines of data in the file
 - It reads all rows (lines) and retains the newlines (換行) character that is at the end of every row (line)

```
file_name = "test.txt"
myfile = open(file_name, "r")
text3 = myfile.readline()      # read the first file as a string
text4 = myfile.readline()      # read the second line from the file
myfile.close()
myfile = open(file_name, "r")
listLines = myfile.readlines() # read file into a list of strings
print(listLines) # ['Line 1\n', 'Line 2\n', 'Line 3\n']
myfile.close()
```


Example: readline()

- Read a file containing multiple lines
- Store the lines into a list of strings
- Report the number of lines

```
1 lineList = []      # store the lines as a list of strings
2 filename = "ppap.txt"
3 f = open(filename, "r")
4 line = f.readline()
5 while len(line) > 0:
6     print(line, end="")
7     # use strip() to remove the '\n' at the end of a line
8     # append the line to a list
9     lineList.append(line.strip())
10    line = f.readline() # read the next line
11 f.close()
12 print("num of lines:", len(lineList))
13 print(lineList)
```

I have a pen I have a Apple, UH! Apple-pen.
I have a pen I have Pineapple, UH! Pineapple-pen.
Apple-pen, Pineapple-pen UH!
Pen-Pineapple-Apple-pen
num of lines: 4
['I have a pen I have a Apple, UH! Apple-pen.', 'I have a pen I have Pineapple, UH! Pineapple-pen.', 'Apple-pen, Pineapple-pen UH!', 'Pen-Pineapple-Apple-pen']

Example: read()

- Read a file containing multiple lines
- Store the lines into a list of strings
- Report the number of lines

```
I have a pen I have a Apple, UH! Apple-pen.  
I have a pen I have Pineapple, UH! Pineapple-pen.  
Apple-pen, Pineapple-pen UH!  
Pen-Pineapple-Apple-pen
```

```
num of lines: 5
```

```
['I have a pen I have a Apple, UH! Apple-pen.', 'I have a  
pen I have Pineapple, UH! Pineapple-pen.', 'Apple-pen,  
Pineapple-pen UH!', 'Pen-Pineapple-Apple-pen', '']
```

```
1 filename = "ppap.txt"  
2 f = open(filename, "r")  
3 # read all lines into a big text string  
4 ftext = f.read()  
5 f.close()  
6 print(ftext)  
7 # split the string into lines using split() with '\n'  
8 lineList = ftext.split("\n")  
9 print("num of lines:", len(lineList))  
10 print(lineList)
```

Example: readlines()

- Read a file containing multiple lines
- Store the lines into a list of strings
- Report the number of lines

```
1 filename = "ppap.txt"
2 f = open(filename, "r")
3 # read all lines in to a list of strings
4 linelist = f.readlines()
5 f.close()
6 print("num of lines:", len(linelist))
7 print(linelist)
```

num of lines: 4

['I have a pen I have a Apple, UH! Apple-pen.\n', 'I have a pen I have Pineapple, UH! Pineapple-pen.\n', 'Apple-pen, Pineapple-pen UH!\n', 'Pen-Pineapple-Apple-pen\n']

File Iterators

- A more convenient way: use **for loop** to step through the file line by line
- **For each iteration** the **variable line** takes on value of a line from the file and the loop body executed

```
file_name = "test.txt"
myfile = open(file_name, "r")
for line in myfile:
    # do something here
myfile.close()
```

Example: File Iterator

- Read a file containing multiple lines
- Store the lines into a list of strings
- Report the number of lines

```
1 filename = "ppap.txt"
2 lineList = []
3 f = open(filename, "r")
4 # use for loop (file iterator) to visit the file line by line
5 for line in f:
6     # at each iteration, the variable line stores a line
7     # remove the '\n' by the built-in string function strip()
8     # append the cleaned line to a list
9     lineList.append(line.strip())
10 f.close()
11 print("num of lines:", len(lineList))
12 print(lineList)
```

num of lines: 4

['I have a pen I have a Apple, UH! Apple-pen.', 'I have a pen I have Pineapple, UH! Pineapple-pen.', 'Apple-pen, Pineapple-pen UH!', 'Pen-Pineapple-Apple-pen']

In-Class Exercise: Word Counter

- Write a program (wordcount.py) that
 - Read a text file (us2016.txt)
 - Count the number of words in each line
 - Count the total number of words in the file

Use `strip()` to remove redundant whitespaces and newlines (`'\n'`)

```
1 filename = "us2016.txt"
2 lineList = []
3 f = open(filename, "r")
4 for line in f:
5     lineList.append(line.strip())
6 f.close()
7
8 i, n = 0, 0
9 while i < len(lineList):
10     line = lineList[i]
11     words = line.split(" ")
12     print("Line", (i+1), "has", len(words), "words")
13     n = n + len(words)
14     i = i + 1
15 print("Total words:", n)
```

```
Line 1 has 20 words
Line 2 has 58 words
Line 3 has 20 words
Line 4 has 69 words
Line 5 has 48 words
Total words: 215
```

Alternative File Reading Method:

with open() **as** **f**: 

- The **with** statement closes the file automatically
 - It is a more Pythonic style



```
1 filename = "ppap.txt"
2 lineList = []
3 f = open(filename, "r")
4 for line in f:
5     lineList.append(line.strip())
6 f.close()
7 print(lineList)
```

Line 2 below combines Line 1, 3, 6 above

```
1 lineList = []
2 with open("ppap.txt", "r") as f:
3     for line in f:
4         lineList.append(line.strip())
5 print(lineList)
```

File-Writing Operations

- Functions available for writing the files:
`write()` and `writelines()`
 - Use the mode **"w"** to write to a file
 - Existing content in the file will be erased
- **`write()`** – write a fixed sequence of characters to a file
 - Do NOT automatically put the newline character at the end of the string it writes. These need to be put manually!

```
file_name = "test.txt"
myfile = open(file_name, "w") 
myfile.write("Hello World!\n") # write "Hello World!\n" into myfile
myfile.write("How are you?\n") # write "How are you?\n" into myfile
myfile.close()
# in test.txt:
Hello World!
How are you?
```


File-Writing Operations

- `writelines()` -- write a list of strings to a file
 - Do NOT automatically put the newline character at the end of the string it writes. These need to be put manually!

```
file_name = "test.txt"   
myfile = open(file_name, "w")  
lines_of_text1 = ["line1\n", "line2\n", "line3\n"]  
lines_of_text2 = ["line4", "line5", "line6"]  
myfile.writelines(lines_of_text1)  
myfile.writelines(lines_of_text2)  
myfile.close()  
# in test.txt:  
line 1  
line 2  
line 3  
line 4line5line 6
```

Example: Writing Files

- Write a program that write a table of powers from 1 to LIMIT into a file (table-of-powers.txt)

```
1 f = open("table-of-powers.txt", "w")
2 LIMIT = 10
3 header = "Table of Powers from 1 to %d\n" % LIMIT
4 bar = "=" * len(header) + "\n\n"
5 f.write(header)
6 f.write(bar)
7 # use loop to generate the powers from 1 to LIMIT
8 n = 1
9 while n < LIMIT + 1:
10     f.write("%3d  %5d  %5d\n" % (n, n**2, n**3))
11     n = n + 1
12 f.close()
```

```
1 Table of Powers from 1 to 10
2 =====
3
4 1      1      1
5 2      4      8
6 3      9     27
7 4     16     64
8 5     25    125
9 6     36    216
10 7     49    343
11 8     64    512
12 9     81    729
13 10    100   1000
14
```

File Reading: `tell()` & `seek()`

- `f.read(size)` reads at most the next `size` characters
 - If size is not specified, the whole file is read
 - Return empty string if no more bytes available
- `f.seek(offset)`: jumps to position `offset` in the file
- `f.tell()`: give the current position in the file 

```
f = open("test.txt", "r")
```

```
text = f.read()
```

```
print(text)
```

```
print("The current position is", f.tell())
```

```
f.seek(0) # jump to the first position of the file
```

```
s = f.read(10)
```

```
print("The current position is", f.tell())
```

```
 while s: # read 10 characters every time
```

```
    print(s)
```

```
    s = f.read(10)
```

```
f.close()
```

text.txt : This is a test.

Output

This is a test.

The current position is 17

The current position is 10

This is a
test.

Example: Search in a File

- Given a pattern, search whether a text file contains such pattern. If yes, write the line number, column number, and the line text into a file

```
1 # you can allow the user to input the pattern to be searched
2 pattern = "Pineapple-pen"
3 fr = open("ppap.txt", "r")
4 line_num = 0
5 result = []
6 for line in fr:
7     # search for the pattern in each line
8     if line.find(pattern) != -1:
9         # if the line contains the pattern, find its index
10        col_num = line.index(pattern)
11        found = "[l-%d, c-%d]: %s" % (line_num, col_num, line)
12        result = result + [found]
13        line_num = line_num + 1
14 fr.close()
15
16 fw = open("ppap_search.txt", "w")
17 fw.writelines(result)
18 fw.close()
```

[l-0, c-35]: I have a pen I have Pineapple, UH! Pineapple-pen.
[l-1, c-11]: Apple-pen, Pineapple-pen UH!

Example: Appending to a File

- Use file mode **"a"** to **append texts to the end of a file**
- Each write adds an fresh line tacked on to the end of the existing file contents (i.e. **does not erase old file contents**)



```
1 msg = input("To be logged: ")
2 while msg:
3     f = open("log.txt", "a")
4     f.write(msg + "\n")
5     msg = input("To be logged: ")
6     f.close()
```

Each time you execute this code, you input is **always appended into the end of log.txt**



```
1 f = open("log.txt", "w")
2 msg = input("To be logged: ")
3 while msg:
4     f.write(msg + "\n")
5     msg = input("To be logged: ")
6 f.close()
```

Each time you execute this code, **log.txt is re-created so that new inputs are written**

Alternative File Writing Method:

with open() **as** f:

```
1 f = open("log.txt", "w")
2 msg = input("To be logged: ")
3 while msg:
4     f.write(msg + "\n")
5     msg = input("To be logged: ")
6 f.close()
```

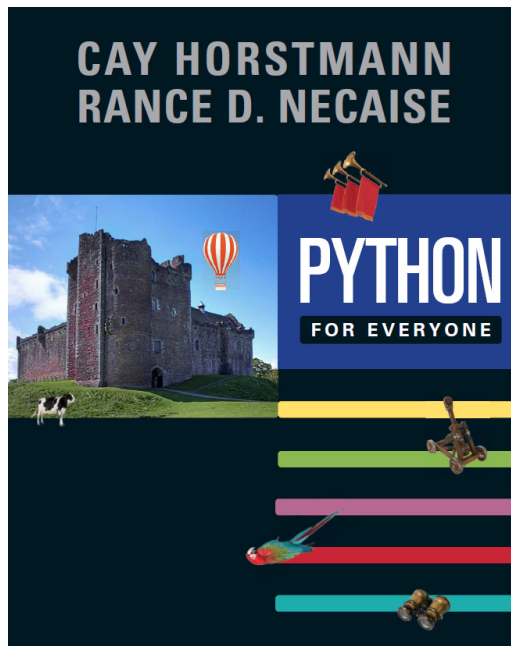
Line 1 below is equal to Line 1 & 6 above

```
1 with open("log.txt", "w") as f:
2     msg = input("To be logged: ")
3     while msg:
4         f.write(msg + "\n")
5         msg = input("To be logged: ")
```

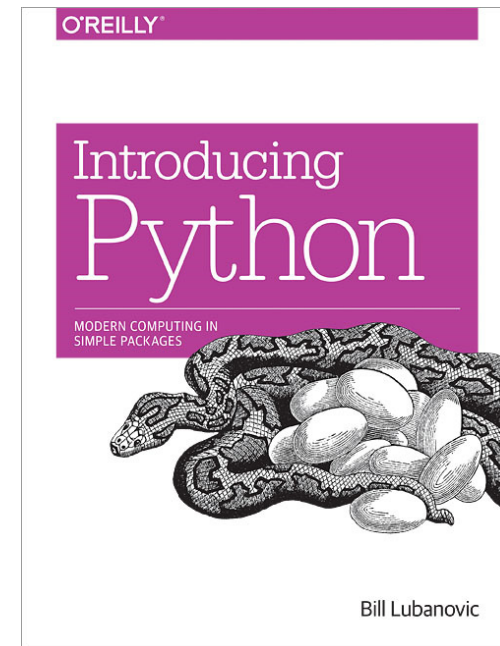
Summary

- File Reading
- File Iterators
- File Writing
- File Search and Appending

Suggested Reading



P.341 – P.354



P.173 – P.180

print()

```
# print() automatically outputs "new line (\n)" at the end  
# of the line by default. If you do not give print()  
# any string to print out, print() will output a "\n"  
print()
```

```
# you can change the default "\n" of the print() by  
# using end= at the last parameter
```

```
print("Hello World") # print("Hello World", end="\n")
```

```
print("Hello World", end="\n\n")
```

```
print("Hello World", end="~~~\t")
```

```
s = "hello"
```

```
i = 0
```

```
while i < len(s):
```

```
    print(s[i], end="    ")
```

```
    i = i + 1
```

```
print()
```



h e l l o

print()

```
# print() automatically use a whitespace " " to separate  
# its different parameters by default.  
# you can change the default " " separation  
# by using the parameter sep= at the last parameter
```

```
x, y, z = 5, 10, 20
```

```
print(x, y, z)           # 5 10 20
```

```
print(x, y, z, sep=" ")  # 5 10 20
```

```
print(x, y, z, sep="")    # 51020
```

```
print(x, y, z, sep=",")   # 5,10,20
```

```
print(x, y, z, sep=",,,") # 5,,,10,,,20
```

```
s = "hello"
```

```
i = 1
```

```
while i < len(s):
```

```
    print(s[i-1], s[i], sep=">>", end=" ")
```

```
    print(",", end="")
```

```
    i = i + 1
```

```
print()
```

```
h>>e ,e>>l ,l>>l ,l>>o ,
```